Gigi Bernstein
Z23652485

Problem 1

```python
from numbers import Real
from typing import Sequence
from typing import List
from typing import Union
from typing import overload
from testif import testif

class NVector:
    def __init__(self, *num_seq) -> None:
        ''' create a vector from either a sequence or seperate numbers
        preconditions:
            - must have at least one argument
            - must contain only real numbers
        postconditions:
            - the vector is created with the given elements'''
        num_list = []
        if len(num_seq) == 0:
            raise TypeError("must have at least one argument")
        if len(num_seq) == 1:
            first_arg = num_seq[0]
            if isinstance(first_arg, (list, tuple)): # if input is list or tuple
                self._elements: List[Real] = list(first_arg)
            else:
                self._elements = [first_arg]
        else: # more than one argument was given
            self._elements = list(num_seq)

        for element in self._elements:
            if not isinstance(element, Real):
                raise TypeError("all elements must be a real number")

    def __len__(self) -> int:
        '''returns the length of the vector.'''
        return len(self._elements)

    def __getitem__(self, index) -> int:
        '''return item at index, negative numbers work like standard lists
        parameters:
            index - the integer of the requested elements place
        raises:
            TypeError: if index is not valid list index
            IndexError: if index is outside valid range of list'''
        return self._elements[index]
```

```
    def __setitem__(self, index, value) -> None:
        '''set item at index, negative numbers work like standard lists
        parameters:
            index - the integer of the requested elements place
            value - the new value to set at the given index
        raises:
            TypeError: if index is not valid list index or value is not a real number
            IndexError: if index is outside valid range of list'''
        if not isinstance(value, Real):
            raise TypeError("value must be a real number")
        self._elements[index] = value

    def __str__(self) -> str:
        '''returns a string representation of the vector.
        parameters:
            none
        returns:
            str - the string representation of the vector'''
        return str(self._elements)

    def __eq__(self, other) -> bool:
        '''returns True if the vectors are equal, False otherwise.
        parameters:
            other - the other vector to compare with
        returns:
            bool - True if the vectors are equal, False otherwise'''
        if not isinstance(other, NVector):
            return False
        return self._elements == other._elements

    def __ne__(self, other) -> bool:
        '''returns True if the vectors are not equal, False otherwise.
        parameters:
            other - the other vector to compare with
        returns:
            bool - True if the vectors are not equal, False otherwise'''
        return not self.__eq__(other)

    def __add__(self, other) -> "NVector":
        '''returns the sum of two vectors.
        parameters:
            other - the other vector to add
        returns:
            NVector - the sum of the two vectors'''
        if not isinstance(other, NVector):
            raise TypeError("other must be an NVector")
        if len(self._elements) != len(other._elements):
            raise ValueError("vectors must have the same length")
        return NVector(*[a + b for a, b in zip(self._elements, other._elements)])
```

```
    def __radd__(self, other) -> "NVector":
        '''returns the sum of two vectors from the right.
        parameters:
            other - the other vector to add
        returns:
            NVector - the sum of the two vectors'''
        return self.__add__(other)

    def __mul__(self, scalar) -> "NVector":
        '''returns the scalar multiplication of the vector.
        parameters:
            scalar - the scalar to multiply the vector by
        returns:
            NVector - the resulting vector after scalar multiplication'''
        if not isinstance(scalar, Real):
            raise TypeError("scalar must be a real number")
        return NVector(*[a * scalar for a in self._elements])

    def __rmul__(self, scalar) -> "NVector":
        '''returns the scalar multiplication of the vector from the right.
        parameters:
            scalar - the scalar to multiply the vector by
        returns:
            NVector - the resulting vector after scalar multiplication'''
        return self.__mul__(scalar)

    def zeros(self, n: int) -> "NVector":
        '''returns a vector of zeros with the specified length.
        parameters:
            n - the length of the vector
        returns:
            NVector - the vector of zeros'''
        if not isinstance(n, int) or n <= 0:
            raise ValueError("n must be a positive integer")
        return NVector(*[0] * n)

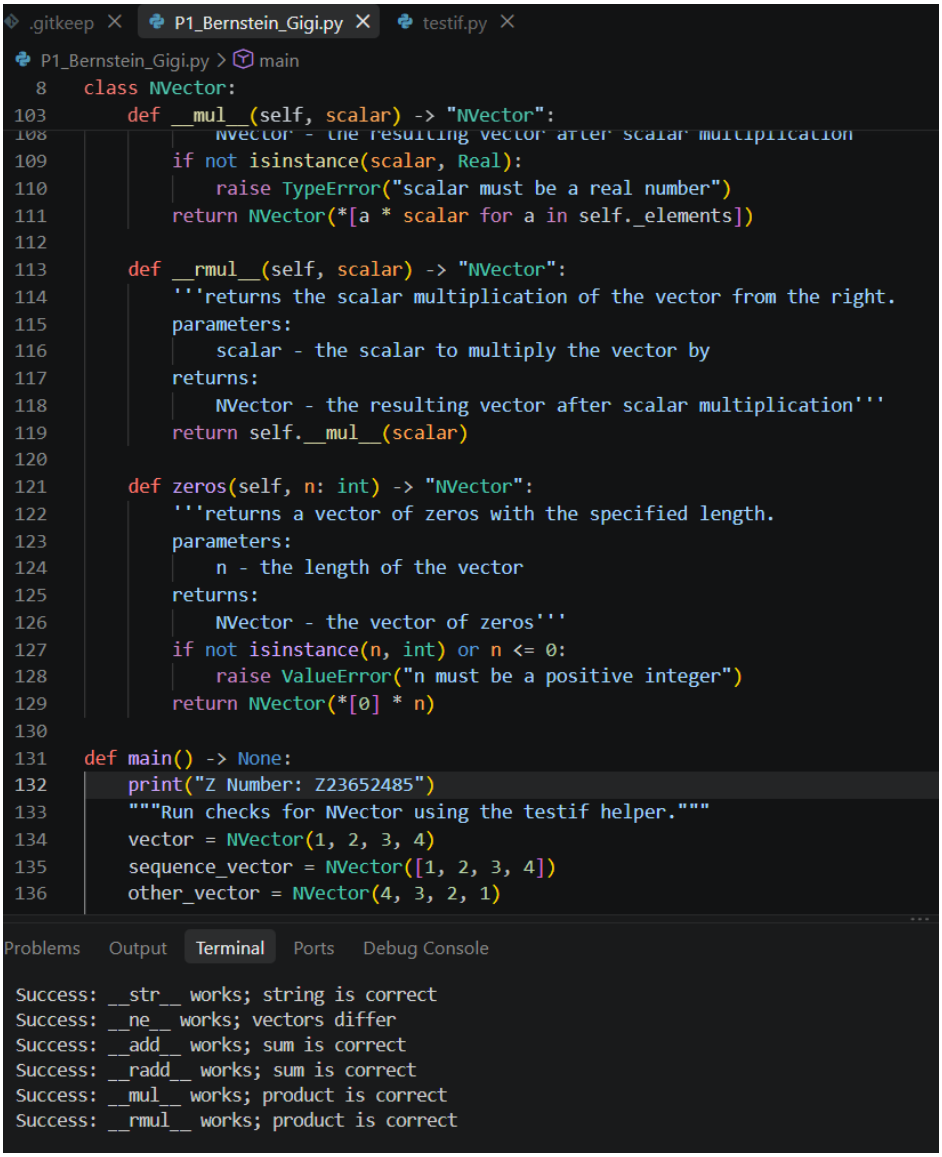
def main() -> None:
    print("Z Number: Z23652485")
    """Run checks for NVector using the testif helper."""
    vector = NVector(1, 2, 3, 4)
    sequence_vector = NVector([1, 2, 3, 4])
    other_vector = NVector(4, 3, 2, 1)

    testif(vector == sequence_vector, "sequence constructor", "vectors match", "vectors do
not match")
    testif(len(vector) == 4, "__len__ works", "length is 4", "length is not 4")
    testif(vector[2] == 3, "__getitem__ works", "index 2 is 3", "index 2 is not 3")

    vector[3] = 10
    testif(vector[3] == 10, "__setitem__ works", "index 3 is 10", "index 3 was not updated")
```

```
    testif(str(vector) == "[1, 2, 3, 10]", "__str__ works", "string is correct", "string is
incorrect")
    testif(vector != other_vector, "__ne__ works", "vectors differ", "vectors should differ")
    testif(vector + other_vector == NVector(5, 5, 5, 11), "__add__ works", "sum is correct",
"sum is incorrect")
    testif(vector.__radd__(other_vector) == NVector(5, 5, 5, 11), "__radd__ works", "sum is
correct", "sum is incorrect")
    testif(vector * 2 == NVector(2, 4, 6, 20), "__mul__ works", "product is correct",
"product is incorrect")
    testif(2 * vector == NVector(2, 4, 6, 20), "__rmul__ works", "product is correct",
"product is incorrect")
    testif(vector.zeros(3) == NVector(0, 0, 0), "zeros works", "three zeros returned",
"incorrect zero vector")


if __name__ == "__main__":
    main()
```



Problem 2

```
class ParkingPermit:
    '''Base class for parking permits.
    attributes:
        owner_name (str): name of permit owner
        months_purchased (int): number of months the permit is valid for
        fee_schedule (dict): dictionary of fees associated with the permit
    methods:
        cost() -> float: calculates the total cost of the permit based on fee schedule
        __str__() -> str: returns a string representation of the permit
        __repr__() -> str: returns a detailed string representation of the permit
    '''
    def __init__(self, owner_name: str, months_purchased: int, fee_schedule: dict) -> None:
        '''initializes a ParkingPermit instance.
        parameters:
            owner_name (str): name of permit owner
            months_purchased (int): number of months the permit is valid for
            fee_schedule (dict): dictionary of fees associated with the permit
        '''
        self.owner_name = owner_name
        self.months_purchased = months_purchased
        self.fee_schedule = fee_schedule

    def cost(self) -> float:
        '''calculates the total cost of the permit based on the fee schedule.
        parameters:
            none
        returns:
            float - the total cost of the permit'''
        total = 0
        for fee_type, fee_amount in self.fee_schedule.items():
            total += fee_amount
        return total

    def __str__(self) -> str:
        '''returns a string representation of the permit.
        parameters:
            none
        returns:
            str - a string representation of the permit'''
        return f"ParkingPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule})"

    def __repr__(self) -> str:
        '''returns a detailed string representation of the permit.
```

```
        parameters:
            none
        returns:
            str - a detailed string representation of the permit'''
        return f"ParkingPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule})"

class StudentPermit(ParkingPermit):
    '''class representing a student parking permit
    inherits from ParkingPermit
    attributes:
        num_vehicles (int): number of vehicles covered by the student permit'''
    def __init__(self, owner_name: str, months_purchased: int, fee_schedule: dict,
num_vehicles: int) -> None:
        '''initializes a StudentPermit instance.
        parameters:
            owner_name (str): name of the permit owner
            months_purchased (int): number of months the permit is valid
            fee_schedule (dict): dictionary of fees associated with the permit
            num_vehicles (int): number of vehicles covered by the student permit
        returns:
            none'''
        super().__init__(owner_name, months_purchased, fee_schedule)
        self.num_vehicles = num_vehicles

    def cost(self) -> float:
        '''calculates the total cost of the student permit.
        parameters:
            none
        returns:
            float - total cost of the student permit'''
        total = super().cost()
        total += 25 * self.months_purchased
        if self.num_vehicles > 1:
            total += 10 * (self.num_vehicles - 1)
        return total

    def __str__(self) -> str:
        '''returns a string representation of the student permit.
        parameters:
            none
        returns:
            str - a string representation of the student permit'''
        return f"StudentPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule},
num_vehicles={self.num_vehicles})"

    def __repr__(self) -> str:
        '''returns a detailed string representation of the student permit.
        parameters:
```

```
            none
        returns:
            str - a detailed string representation of the student permit'''
        return f"StudentPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule},
num_vehicles={self.num_vehicles})"

class FacultyPermit(ParkingPermit):
    '''class representing a faculty parking permit
    inherits from ParkingPermit
    attributes:
        garage_access (bool): indicates if the faculty permit includes garage access'''
    def __init__(self, owner_name: str, months_purchased: int, fee_schedule: dict,
garage_access: bool) -> None:
        '''initializes a FacultyPermit instance.
        parameters:
            owner_name (str): name of the permit owner
            months_purchased (int): number of months the permit is valid
            fee_schedule (dict): dictionary of fees associated with the permit
            garage_access (bool): indicates if the faculty permit includes garage access
        returns:
            none'''
        super().__init__(owner_name, months_purchased, fee_schedule)
        self.garage_access = garage_access

    def cost(self) -> float:
        '''calculates the total cost of the faculty permit.
        parameters:
            none
        returns:
            float - total cost of the faculty permit'''
        total = super().cost()
        total += 40 * self.months_purchased
        if self.garage_access:
            total += 15 * self.months_purchased
        return total

    def __str__(self) -> str:
        '''returns a string representation of the faculty permit.
        parameters:
            none
        returns:
            str - a string representation of the faculty permit'''
        return f"FacultyPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule},
garage_access={self.garage_access})"

    def __repr__(self) -> str:
        '''returns a detailed string representation of the faculty permit.
        parameters:
```

```
            none
        returns:
            str - a detailed string representation of the faculty permit'''
        return f"FacultyPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule},
garage_access={self.garage_access})"

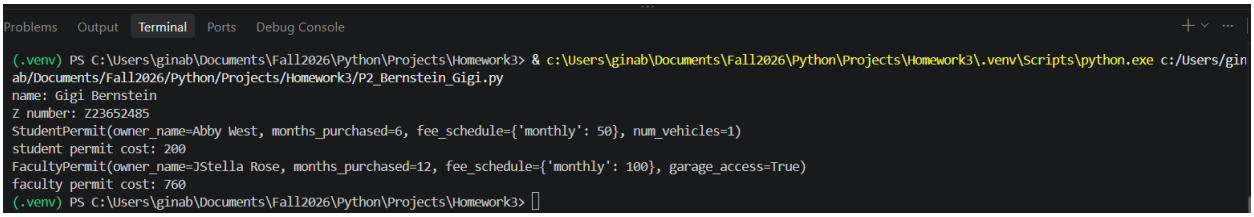
def main():
    '''main function to demonstrate the use of StudentPermit and FacultyPermit classes.'''
    student_permit = StudentPermit("Abby West", 6, {"monthly": 50}, 1)
    faculty_permit = FacultyPermit("JStella Rose", 12, {"monthly": 100}, True)

    print("name: Gigi Bernstein")
    print("Z number: Z23652485")

    print(student_permit)
    print(f"student permit cost: {student_permit.cost()}")

    print(faculty_permit)
    print(f"faculty permit cost: {faculty_permit.cost()}")

if __name__ == "__main__":
    main()
```



Problem 3

```
'''predator-prey-human simulation on a 2d island grid.'''

from __future__ import annotations

import random
from typing import Type

import matplotlib.pyplot as plt


class Island:
    '''n-by-n grid that stores prey, predators, and humans'''

    def __init__(self, n: int, prey_count: int = 0, predator_count: int = 0, count_humans:
int = 0) -> None:
```

```
        '''create an island and randomly place the initial population
        preconditions:
            - n is positive;
            - all population counts are nonnegative;
            - their sum does not exceed grid capacity
        postconditions:
            - grid contains the requested number of each class'''
        if n <= 0:
            raise ValueError("island size must be positive")
        populations = (prey_count, predator_count, count_humans)
        if any(count < 0 for count in populations):
            raise ValueError("population counts cannot be negative")
        if sum(populations) > n * n:
            raise ValueError("initial population exceeds island capacity")

        self.grid_size: int = n
        self.grid: list[list[Animal | int]] = [[0 for _ in range(n)] for _ in range(n)]
        self.init_animals(prey_count, predator_count, count_humans)

    def init_animals(
        self, prey_count: int, predator_count: int, count_humans: int
    ) -> None:
        '''randomly place requested numbers of prey, predators, and humans
        paramaters:
            - prey_count: number of prey to place
            - predator_count: number of predators to place
            - count_humans: number of humans to place
        returns: None'''
        self._place_random_animals(prey_count, Prey)
        self._place_random_animals(predator_count, Predator)
        self._place_random_animals(count_humans, Human)

    def _place_random_animals(self, count: int, animal_class: Type[Animal]) -> None:
        '''place count instances of animal_class in randomly selected empty cells
        paramaters:
            - count: number of animals to place
            - animal_class: the class of animals to place
        returns: None'''
        placed = 0
        while placed < count:
            x = random.randrange(self.grid_size)
            y = random.randrange(self.grid_size)
            if not self.animal(x, y):
                self.register(animal_class(island=self, x=x, y=y))
                placed += 1

    def clear_all_moved_flags(self) -> None:
        '''reset every animal's moved flag before a new time unit begins
        paramaters: None
        returns: None'''
```

```
        for column in self.grid:
            for occupant in column:
                if isinstance(occupant, Animal):
                    occupant.clear_moved_flag()

    def size(self) -> int:
        '''return the length of one side of the square island
        paramaters: None
        returns: int'''
        return self.grid_size

    def register(self, animal: Animal) -> None:
        '''place animal in the grid cell matching its current coordinates
        paramaters:
            - animal: the animal to place
        returns: None'''
        self.grid[animal.x][animal.y] = animal

    def remove(self, animal: Animal) -> None:
        '''empty animal's current grid cell if that cell still contains it
        paramaters:
            - animal: the animal to remove
        returns: None'''
        if self.grid[animal.x][animal.y] is animal:
            self.grid[animal.x][animal.y] = 0

    def animal(self, x: int, y: int) -> Animal | int:
        '''return the occupant at (x, y), or -1 when coordinates are out of bounds
        paramaters:
            - x: the x-coordinate
            - y: the y-coordinate
        returns: Animal | int'''
        if 0 <= x < self.grid_size and 0 <= y < self.grid_size:
            return self.grid[x][y]
        return -1

    def __str__(self) -> str:
        '''return a grid display with the island's lower-left cell at the bottom
        paramaters: None
        returns: str'''
        rows: list[str] = []
        for y in range(self.grid_size - 1, -1, -1):
            row = []
            for x in range(self.grid_size):
                occupant = self.grid[x][y]
                row.append(str(occupant) if isinstance(occupant, Animal) else ".")
            rows.append(" ".join(row))
        return "\n".join(rows)

    def count_prey(self) -> int:
```

```
        '''return the number of Prey objects currently on the grid
        paramaters: None
        returns: int'''
        return sum(
            isinstance(occupant, Prey)
            for column in self.grid
            for occupant in column
        )

    def count_predators(self) -> int:
        '''return the number of Predator objects currently on the grid
        paramaters: None
        returns: int'''
        return sum(
            isinstance(occupant, Predator)
            for column in self.grid
            for occupant in column
        )

    def count_humans(self) -> int:
        '''return the number of Human objects currently on the grid
        paramaters: None
        returns: int'''
        return sum(
            isinstance(occupant, Human)
            for column in self.grid
            for occupant in column
        )


class Animal:
    '''Base class for entities that move and breed on an Island.'''

    breed_time: int
    breed_clock: int

    def __init__(self, island: Island, x: int = 0, y: int = 0, s: str = "A") -> None:
        '''initialize an animal at (x, y) on island with display symbol s
        paramaters:
            - island: the island on which the animal resides
            - x: the x-coordinate
            - y: the y-coordinate
            - s: the display symbol
        returns: None'''
        self.island: Island = island
        self.name: str = s
        self.x: int = x
        self.y: int = y
        self.moved: bool = False
```

```
    def position(self) -> tuple[int, int]:
        '''return the animal's current (x, y) coordinates
        paramaters: none
        returns: tuple[int, int]'''
        return self.x, self.y

    def __str__(self) -> str:
        '''return the animal's one-character grid symbol
        paramaters: none
        returns: str'''
        return self.name

    def check_grid(self, type_looking_for: type = int) -> tuple[int, int] | int:
        '''return a neighboring cell containing the exact requested type, or 0
        paramaters:
            - type_looking_for: the type of animal to look for
        returns: tuple[int, int] | int'''
        offsets = ((-1, 1), (0, 1), (1, 1), (-1, 0), (1, 0), (-1, -1), (0, -1), (1, -1))
        for dx, dy in offsets:
            x = self.x + dx
            y = self.y + dy
            if 0 <= x < self.island.size() and 0 <= y < self.island.size():
                if type(self.island.animal(x, y)) is type_looking_for:
                    return x, y
        return 0

    def _move_to(self, location: tuple[int, int]) -> None:
        '''move this animal into location and mark it as moved for this time unit
        paramaters:
            - location: the (x, y) coordinates to move to
        returns: none'''
        self.island.remove(self)
        self.x, self.y = location
        self.island.register(self)
        self.moved = True

    def move(self) -> None:
        '''move to the first neighboring empty cell unless already moved
        paramaters: none
        returns: None'''
        if not self.moved:
            location = self.check_grid(int)
            if isinstance(location, tuple):
                self._move_to(location)

    def breed(self) -> None:
        '''create an offspring in a neighboring empty cell when breed_clock expires
        paramaters: none
        returns: none'''
        if self.breed_clock <= 0:
```

```
            location = self.check_grid(int)
            if isinstance(location, tuple):
                self.breed_clock = self.breed_time
                offspring = type(self)(self.island, x=location[0], y=location[1])
                self.island.register(offspring)

    def clear_moved_flag(self) -> None:
        '''mark the animal as not yet moved during the next time unit
        paramaters: none
        returns: none'''
        self.moved = False


class Prey(Animal):
    '''An animal that moves and breeds but does not eat other animals.'''

    breed_time: int = 3

    def __init__(self, island: Island, x: int = 0, y: int = 0, s: str = "O") -> None:
        '''create prey and initialize its breeding clock from Prey.breed_time
        paramaters:
            - island: the island on which the prey resides
            - x: the x-coordinate
            - y: the y-coordinate
            - s: the display symbol
        returns: None'''
        super().__init__(island, x, y, s)
        self.breed_clock = self.breed_time

    def clock_tick(self) -> None:
        '''advance the breeding clock by one time unit
        paramaters: none
        returns: none'''
        self.breed_clock -= 1


class Predator(Animal):
    '''An animal that eats prey and starves when it cannot eat in time.'''

    breed_time: int = 6
    starve_time: int = 3

    def __init__(self, island: Island, x: int = 0, y: int = 0, s: str = "X") -> None:
        '''create a predator and initialize breeding and starvation clocks
        paramaters:
            - island: the island on which the predator resides
            - x: the x-coordinate
            - y: the y-coordinate
            - s: the display symbol
        returns: None'''
```

```
        super().__init__(island, x, y, s)
        self.starve_clock: int = self.starve_time
        self.breed_clock = self.breed_time

    def clock_tick(self) -> None:
        '''advance clocks and remove the predator if its starvation clock expires
        paramaters: none
        returns: none'''
        self.breed_clock -= 1
        self.starve_clock -= 1
        if self.starve_clock <= 0:
            self.island.remove(self)

    def eat(self) -> None:
        '''eat a neighboring Prey, move into its cell, and reset starvation
        paramaters: none
        returns: none'''
        if not self.moved:
            location = self.check_grid(Prey)
            if isinstance(location, tuple):
                prey = self.island.animal(*location)
                if isinstance(prey, Prey):
                    self.island.remove(prey)
                    self._move_to(location)
                    self.starve_clock = self.starve_time


class Human(Animal):
    '''An animal that periodically hunts predators and can itself starve.'''

    hunt_time: int = 4
    starve_time: int = 12
    breed_time: int = 8

    def __init__(self, island: Island, x: int = 0, y: int = 0, s: str = "H") -> None:
        '''create a human with clocks initialized from Human's class attributes
        paramaters:
            - island: the island on which the human resides
            - x: the x-coordinate
            - y: the y-coordinate
            - s: the display symbol
        returns: None'''
        super().__init__(island, x, y, s)
        self.hunt_clock: int = self.hunt_time
        self.starve_clock: int = self.starve_time
        self.breed_clock = self.breed_time

    def eat(self) -> None:
        '''hunt at each hunt_time interval, eating a neighboring Predator if found
        paramaters: none
```

```
        returns: none

        A successful hunt moves the human into the predator's cell and resets its
        starvation clock. Prey are never targeted by this method.
        '''
        self.hunt_clock -= 1
        if self.hunt_clock > 0:
            return
        self.hunt_clock = self.hunt_time
        if not self.moved:
            location = self.check_grid(Predator)
            if isinstance(location, tuple):
                predator = self.island.animal(*location)
                if isinstance(predator, Predator):
                    self.island.remove(predator)
                    self._move_to(location)
                    self.starve_clock = self.starve_time

    def clock_tick(self) -> None:
        '''advance human clocks and remove the human if it starves
        paramaters: none
        returns: none'''
        self.breed_clock -= 1
        self.starve_clock -= 1
        if self.starve_clock <= 0:
            self.island.remove(self)


def main(
    predator_breed_time: int = 6,
    predator_starve_time: int = 3,
    initial_predators: int = 10,
    prey_breed_time: int = 3,
    initial_prey: int = 50,
    size: int = 10,
    ticks: int = 1000,
    human_hunt_time: int = 4,
    human_starve_time: int = 12,
    human_breed_time: int = 8,
    initial_humans: int = 2) -> tuple[list[int], list[int], list[int]]:
    '''Run the island simulation, plot populations, and return their histories.
    parameters:
        - the *_time arguments set class clock periods
        - the initial_* arguments set starting populations
        - size is the grid width
        - ticks is capped at 1000
        - set starting populations; size is the grid width; ticks is capped at 1000.
        - simulation stops after 20 time units in a row with unchanged counts or when tick
limit is reached
    returns:
```

```
    - returned lists include time zero.
    '''
    if ticks < 0:
        raise ValueError("ticks cannot be negative")
    if any(clock <= 0 for clock in (
            predator_breed_time,
            predator_starve_time,
            prey_breed_time,
            human_hunt_time,
            human_starve_time,
            human_breed_time,)):
        raise ValueError("all clock periods must be positive")

    Predator.breed_time = predator_breed_time
    Predator.starve_time = predator_starve_time
    Prey.breed_time = prey_breed_time
    Human.hunt_time = human_hunt_time
    Human.starve_time = human_starve_time
    Human.breed_time = human_breed_time

    island = Island(size, initial_prey, initial_predators, initial_humans)
    print("Initial island:")
    print(island)

    prey_history = [island.count_prey()]
    predator_history = [island.count_predators()]
    human_history = [island.count_humans()]
    unchanged_ticks = 0
    stability_window = 20

    # Snapshotting ensures every animal present at tick start acts at most once.
    for tick in range(min(ticks, 1000)):
        island.clear_all_moved_flags()
        animals = [
            occupant
            for column in island.grid
            for occupant in column
            if isinstance(occupant, Animal)
        ]
        for animal in animals:
            if island.animal(animal.x, animal.y) is not animal:
                continue
            if isinstance(animal, (Predator, Human)):
                animal.eat()
            animal.move()
            animal.breed()
            animal.clock_tick()

        current_counts = (
            island.count_prey(),
```

```
            island.count_predators(),
            island.count_humans(),
        )
        previous_counts = (
            prey_history[-1],
            predator_history[-1],
            human_history[-1],
        )
        prey_history.append(current_counts[0])
        predator_history.append(current_counts[1])
        human_history.append(current_counts[2])
        unchanged_ticks = unchanged_ticks + 1 if current_counts == previous_counts else 0

        if tick % 10 == 0:
            print(f"Tick {tick + 1}: prey={current_counts[0]}, predators={current_counts[1]},
humans={current_counts[2]}")
        if unchanged_ticks >= stability_window:
            print(f"Populations remained constant for {stability_window} ticks.")
            break

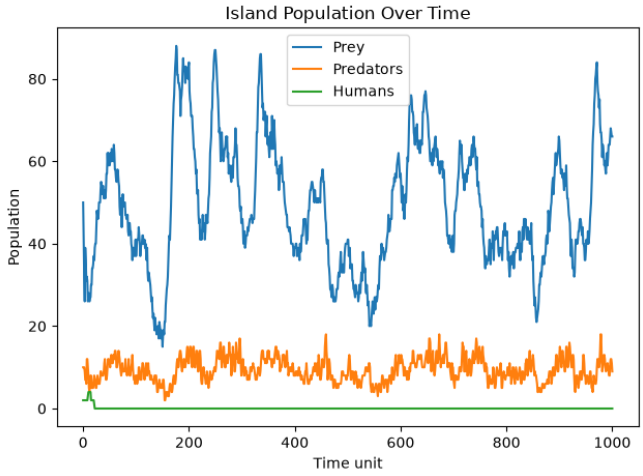
    print("name: Gigi Bernstein")
    print("Z number: Z23652485")

    print("Final island:")
    print(island)

    plt.plot(prey_history, label="Prey")
    plt.plot(predator_history, label="Predators")
    plt.plot(human_history, label="Humans")
    plt.xlabel("Time unit")
    plt.ylabel("Population")
    plt.title("Island Population Over Time")
    plt.legend()
    plt.tight_layout()
    plt.show()
    return prey_history, predator_history, human_history


if __name__ == "__main__":
    main()
```

```
(.venv) PS C:\Users\ginab\Documents\Fall202
ab/Documents/Fall2026/Python/Projects/Homew
Initial island:
. O O . O O . . . .
H O . . . . O O . O
. O . . O O . X . O
. . . X O . O O O X
X . O O O O . X . O
X O O O O . . O O O
. X . O O O O O . O
. . H O O . O . X X
O . . O O O O . O O
X . O . O O O O . O
Tick 1: prey=41, predators=10, humans=2
Tick 11: prey=28, predators=5, humans=4
Tick 21: prey=36, predators=8, humans=1
Tick 31: prey=50, predators=6, humans=0
Tick 41: prey=53, predators=12, humans=0
Tick 51: prey=60, predators=10, humans=0
Tick 61: prey=58, predators=14, humans=0
Tick 71: prey=51, predators=9, humans=0
Tick 81: prey=47, predators=9, humans=0
Tick 91: prey=40, predators=10, humans=0
Tick 101: prey=39, predators=8, humans=0
Tick 111: prey=40, predators=9, humans=0
Tick 121: prey=39, predators=8, humans=0
Tick 131: prey=24, predators=7, humans=0
Tick 141: prey=20, predators=4, humans=0
Tick 151: prey=18, predators=7, humans=0
Tick 161: prey=36, predators=3, humans=0
Tick 171: prey=74, predators=8, humans=0
Tick 181: prey=79, predators=11, humans=0
Tick 191: prey=83, predators=12, humans=0
Tick 201: prey=77, predators=14, humans=0
Tick 211: prey=61, predators=10, humans=0
Tick 221: prey=41, predators=14, humans=0
Tick 231: prey=43, predators=7, humans=0
Tick 241: prey=64, predators=8, humans=0
Tick 251: prey=85, predators=10, humans=0
Tick 261: prey=66, predators=12, humans=0
Tick 271: prey=65, predators=13, humans=0
Tick 281: prey=57, predators=9, humans=0
Tick 291: prey=61, predators=13, humans=0
Tick 981: prey=61, predators=9, humans=0
Tick 991: prey=59, predators=11, humans=0
name: Gigi Bernstein
Z number: Z23652485
Final island:
. . . O O X . X . .
X . . . X X O X O .
. O X O X O O O O O
O X O O O O O . . .
O O O O O O O . O O
O O O O O O O . O O
O O O O O O O . O .
O O O O O O O . O .
O O O O O O O . O .
O O O O O O O . . .
```



Problem 4

```
'''simple non-interactive text-file editing commands and demonstrations.'''

from __future__ import annotations

import os
import stat
import tempfile
from pathlib import Path

from testif import testif


def _validate_text(value: str, parameter_name: str) -> None:
    ''' checks if text is string
    parameters:
        value: the text to validate.
        parameter_name: the name of the parameter for error messages
    return:
        none
    raise:
        TypeError: if value is not a string.'''
    if not isinstance(value, str):
        raise TypeError(f"{parameter_name} must be a string")


def _write_atomically(filename: str, content: str) -> None:
    '''replaces filename with content, but leaves the original intact if theres a failure.
    parameters:
        filename: the path to the file to replace
        content: the new content to write to the file
    return:
        none
    raise:
        OSError: if there is an error writing to the temporary file or replacing the original
file
    '''
    path = Path(filename)
    try:
        existing_mode = stat.S_IMODE(path.stat().st_mode)
    except FileNotFoundError:
        existing_mode = None

    with tempfile.TemporaryDirectory(dir=path.parent) as temp_directory:
        temporary_path = Path(temp_directory) / path.name
        temporary_path.write_text(content, encoding="utf-8")
        if existing_mode is not None:
            temporary_path.chmod(existing_mode)
        os.replace(temporary_path, path)
```

```
def ed_read(filename: str, from_pos: int = 0, to: int = -1) -> str:
    '''read and return the file text in the half-open interval [from_pos, to).

    preconditions: \
        - filename is a path string
        -  from_pos and to are integers
        - from_pos is nonnegative
        - to is -1 or nonnegative.
    postconditions:
        - the file is unchanged and the requested text is returned
        - if to is greater than the file length, raise IndexError
        - File I/O errors propagate to the caller
    '''
    _validate_text(filename, "filename")
    if not isinstance(from_pos, int) or isinstance(from_pos, bool):
        raise TypeError("from_pos must be an integer")
    if not isinstance(to, int) or isinstance(to, bool):
        raise TypeError("to must be an integer")
    if from_pos < 0:
        raise ValueError("from_pos cannot be negative")
    if to < -1:
        raise ValueError("to must be -1 or nonnegative")

    content = Path(filename).read_text(encoding="utf-8")
    if to > len(content):
        raise IndexError(f"to ({to}) exceeds file length ({len(content)})")
    end = len(content) if to == -1 else to
    return content[from_pos:end]


def ed_find(filename: str, search_str: str) -> list[int]:
    '''return:
        - list of the non-overlapping start positions of search_str in the file.

    preconditions:
        - filename and search_str are strings.
    postconditions:
        - file is unchanged
        - an empty search string matches each character boundary, including after final
character
        - File I/O errors propagate to the caller
    '''
    _validate_text(filename, "filename")
    _validate_text(search_str, "search_str")
    content = Path(filename).read_text(encoding="utf-8")
    if search_str == "":
        return list(range(len(content) + 1))
```

```
    positions: list[int] = []
    start = 0
    while True:
        position = content.find(search_str, start)
        if position == -1:
            return positions
        positions.append(position)
        start = position + len(search_str)


def ed_replace(
    filename: str,
    search_str: str,
    replace_with: str,
    occurrence: int = -1,
) -> int:
    '''replace all or one indexed non-overlapping occurrence in a file.

    preconditions:
        - filename, search_str, and replace_with are strings
        - occurrence is -1 (replace all) or a nonnegative integer
    postconditions:
        - returns the number of replacements
        - An occurrence index beyond the matches returns 0 without changing the file
        - The file is atomically replaced only after its new contents are ready
        - File I/O errors propagate to the caller
    '''
    _validate_text(filename, "filename")
    _validate_text(search_str, "search_str")
    _validate_text(replace_with, "replace_with")
    if not isinstance(occurrence, int) or isinstance(occurrence, bool):
        raise TypeError("occurrence must be an integer")
    if occurrence < -1:
        raise ValueError("occurrence must be -1 or nonnegative")

    path = Path(filename)
    content = path.read_text(encoding="utf-8")
    positions = (
        list(range(len(content) + 1))
        if search_str == ""
        else ed_find(filename, search_str)
    )

    if occurrence == -1:
        if not positions:
            return 0
        updated_content = content.replace(search_str, replace_with)
        replacement_count = len(positions)
    elif occurrence >= len(positions):
        return 0
```

```
    else:
        position = positions[occurrence]
        updated_content = (
            content[:position]
            + replace_with
            + content[position + len(search_str):]
        )
        replacement_count = 1

    _write_atomically(filename, updated_content)
    return replacement_count


def ed_append(filename: str, string: str) -> int:
    '''Append string to filename, creating the file if it does not exist.

    preconditions:
        - filename and string are strings
    postconditions:
        - returns the number of characters appended
        - File I/O errors propagate to the caller
        - the target is replaced only after the updated contents have been written
successfully
    '''
    _validate_text(filename, "filename")
    _validate_text(string, "string")
    path = Path(filename)
    try:
        original_content = path.read_text(encoding="utf-8")
    except FileNotFoundError:
        original_content = ""
    _write_atomically(filename, original_content + string)
    return len(string)


def test_ed_find() -> bool:
    '''test ed_find's positions, missing matches, and empty-string behavior'''
    with tempfile.TemporaryDirectory() as temp_directory:
        filename = str(Path(temp_directory) / "find_test.txt")
        ed_append(filename, "abc abc ab")
        positions_match = ed_find(filename, "abc") == [0, 4]
        no_match = ed_find(filename, "xyz") == []
        empty_match = ed_find(filename, "") == list(range(11))
        return all((
            testif(positions_match, "ed_find positions", "found both matches", "wrong
positions"),
            testif(no_match, "ed_find missing text", "returned no matches", "unexpected
match"),
            testif(empty_match, "ed_find empty string", "found text boundaries", "wrong
boundaries"),
```

```
        ))


def test_ed_replace() -> bool:
    '''test ed_replace's all, indexed, absent-index, and empty-string cases'''
    with tempfile.TemporaryDirectory() as temp_directory:
        filename = str(Path(temp_directory) / "replace_test.txt")
        ed_append(filename, "one two one")
        replaced_all = ed_replace(filename, "one", "1") == 2
        all_content_correct = ed_read(filename) == "1 two 1"
        replaced_one = ed_replace(filename, "1", "first", 1) == 1
        one_content_correct = ed_read(filename) == "1 two first"
        beyond_matches = ed_replace(filename, "1", "unused", 2) == 0
        unchanged_on_missing = ed_read(filename) == "1 two first"
        empty_replaced = ed_replace(filename, "", "|", 0) == 1
        empty_content_correct = ed_read(filename) == "|1 two first"
        return all((
            testif(replaced_all and all_content_correct, "ed_replace all", "replaced both
matches", "all replacement failed"),
            testif(replaced_one and one_content_correct, "ed_replace occurrence", "replaced
indexed match", "indexed replacement failed"),
            testif(beyond_matches and unchanged_on_missing, "ed_replace missing occurrence",
"left file unchanged", "changed file unexpectedly"),
            testif(empty_replaced and empty_content_correct, "ed_replace empty string",
"replaced at selected boundary", "empty search failed"),
        ))


def main() -> None:
    '''run unit tests and demonstrate each editing command on a temporary file'''
    test_ed_find()
    test_ed_replace()

    with tempfile.TemporaryDirectory() as temp_directory:
        filename = str(Path(temp_directory) / "editor_demo.txt")
        appended = ed_append(filename, "0123456789")
        ed_append(filename, "0123456789")
        print(f"ed_append wrote {appended} characters")
        print(f"ed_read(3, 9): {ed_read(filename, 3, 9)}")
        print(f"ed_read(3): {ed_read(filename, 3)}")
        print(f"ed_find('345'): {ed_find(filename, '345')}")
        print(f"ed_find('356'): {ed_find(filename, '356')}")
        replacements = ed_replace(filename, "345", "ABCDE", 1)
        print(f"ed_replace occurrence 1 replaced {replacements} match")
        print(f"Final file contents: {ed_read(filename)}")
        print("name: Gigi Bernstein")
        print("Z number: Z23652485")


if __name__ == "__main__":
    main()
```

```
(.venv) PS C:\Users\ginab\Documents\Fall2026\Python\Projects\Homework3> & c:\Users\ginab\Documents\Fall20
venv\Scripts\python.exe c:/Users/ginab/Documents/Fall2026/Python/Projects/Homework3/P4_Bernstein_Gigi.py
Success: ed_find positions; found both matches
Success: ed_find missing text; returned no matches
Success: ed_find empty string; found text boundaries
Success: ed_replace all; replaced both matches
Success: ed_replace occurrence; replaced indexed match
Success: ed_replace missing occurrence; left file unchanged
Success: ed_replace empty string; replaced at selected boundary
ed_append wrote 10 characters
ed_read(3, 9): 345678
ed_read(3): 34567890123456789
ed_find('345'): [3, 13]
ed_find('356'): []
ed_replace occurrence 1 replaced 1 match
Final file contents: 0123456789012ABCDE6789
name: Gigi Bernstein
Z number: Z23652485
```

Problem 5

```python
import math
import turtle
from testif import testif


def strB(n: int, base: int = 10) -> str:
    '''return n as a string of digits in base, using A-Z for values above 9'''
    if type(n) is not int or type(base) is not int:
        raise TypeError("n and base must be integers")
    if n < 0:
        raise ValueError("n must be non-negative")
    if not 2 <= base <= 26:
        raise ValueError("base must be between 2 and 26")

    digits = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    if n < base:
        return digits[n]
    return strB(n // base, base) + digits[n % base]


def test_strB() -> bool:
    '''check zero, decimal output, alphabetic digits, and a large conversion'''
    return all((
        testif(strB(0) == "0", "strB zero", "returned 0", "incorrect zero representation"),
```

```
        testif(strB(1234) == "1234", "strB decimal", "returned 1234", "incorrect decimal
representation"),
        testif(strB(1234, 16) == "4D2", "strB hexadecimal", "returned 4D2", "incorrect
hexadecimal representation"),
        testif(strB(123456789, 26) == "AA44A1", "strB base 26", "returned AA44A1", "incorrect
base-26 representation"),
    ))


def Cnk_m(n: int, k: int, memo: dict[tuple[int, int], int] | None = None) -> int:
    '''return n choose k recursively, saving intermediate results in memo'''
    if type(n) is not int or type(k) is not int:
        raise TypeError("n and k must be integers")
    if n < 0 or k < 0 or k > n:
        raise ValueError("n and k must satisfy n >= k >= 0")
    if memo is None:
        memo = {}

    if k == 0 or k == n:
        return 1
    key = (n, k)
    if key not in memo:
        memo[key] = Cnk_m(n - 1, k - 1, memo) + Cnk_m(n - 1, k, memo)
    return memo[key]


def test_Cnk_m() -> bool:
    '''check the binomial coefficient base cases and recursive values'''
    return all((
        testif(Cnk_m(0, 0) == 1, "Cnk_m zero", "returned 1", "incorrect zero case"),
        testif(Cnk_m(8, 0) == 1, "Cnk_m k zero", "returned 1", "incorrect k=0 case"),
        testif(Cnk_m(8, 8) == 1, "Cnk_m k equals n", "returned 1", "incorrect k=n case"),
        testif(Cnk_m(30, 15) == 155117520, "Cnk_m recursive", "returned 155117520",
"incorrect coefficient"),
    ))


def run_Cnk() -> None:
    '''print several binomial coefficients for a console demonstration'''
    for n, k in ((5, 2), (20, 10), (30, 15)):
        print(f"C({n}, {k}) = {Cnk_m(n, k)}")


def make_pairs(seq1: list, seq2: list, accumulator: list | None = None) -> list:
    '''recursively pair corresponding elements until either list ends'''
    if accumulator is None:
        accumulator = []
    if not seq1 or not seq2:
        return accumulator
    accumulator.append((seq1[0], seq2[0]))
    return make_pairs(seq1[1:], seq2[1:], accumulator)
```

```
def test_make_pairs() -> bool:
    '''check paired values, unequal lengths, and empty-list boundaries'''
    return all((
        testif(make_pairs([1, 2, 3], [4, 5, 6]) == [(1, 4), (2, 5), (3, 6)], "make_pairs
equal", "paired all values", "incorrect equal-length pairs"),
        testif(make_pairs([1, 2, 3], [4, 5]) == [(1, 4), (2, 5)], "make_pairs shorter
second", "stopped at second list", "incorrect shorter-second result"),
        testif(make_pairs([1, 2], [4, 5, 6]) == [(1, 4), (2, 5)], "make_pairs shorter first",
"stopped at first list", "incorrect shorter-first result"),
        testif(make_pairs([], [4, 5, 6]) == [], "make_pairs empty first", "returned empty
list", "incorrect empty-first result"),
        testif(make_pairs([1, 2, 3], []) == [] and make_pairs([], []) == [], "make_pairs
empty second and both", "returned empty lists", "incorrect empty-list result"),
    ))


def _leaf_centerline_point(progress: float, blade_length: float, curve_amount: float) ->
tuple[float, float]:
    return curve_amount * math.sin(math.pi * progress), blade_length * progress


def _draw_leaf_vein(
    length: float,
    base_heading: float,
    side: int,
    recursion_level: int,
    curved: bool,
    branch_angle: float = 52,
) -> None:
    start_position = turtle.position()
    turtle.setheading(base_heading + side * branch_angle)

    if curved:
        for _ in range(10):
            turtle.forward(length / 10)
            turtle.left(side * 0.8)
    else:
        turtle.forward(length)

    end_heading = turtle.heading()
    if recursion_level > 1:
        child_length = length * 0.48
        _draw_leaf_vein(child_length, end_heading, -1, recursion_level - 1, curved, 34)
        _draw_leaf_vein(child_length, end_heading, 1, recursion_level - 1, curved, 34)

    turtle.penup()
    turtle.goto(start_position)
    turtle.setheading(base_heading)
    turtle.pendown()
```

```
def _draw_leaf(
    recursion_level: int,
    petiole_length: float,
    curved: bool,
    start_position: tuple[float, float],
) -> None:
    if recursion_level < 1:
        raise ValueError("recursion_level must be at least 1")
    if petiole_length <= 0:
        raise ValueError("petiole_length must be positive")

    turtle.delay(0)
    turtle.penup()
    turtle.goto(start_position)
    turtle.setheading(90)
    turtle.pendown()
    turtle.forward(petiole_length)

    blade_length = petiole_length * 1.5
    curve_amount = blade_length * 0.24 if curved else 0
    blade_start = turtle.position()

    if curved:
        for step in range(1, 41):
            progress = step / 40
            x, y = _leaf_centerline_point(progress, blade_length, curve_amount)
            turtle.goto(blade_start[0] + x, blade_start[1] + y)
    else:
        turtle.forward(blade_length)

    station_count = max(4, recursion_level * 2 + 2)
    for station in range(1, station_count + 1):
        progress = station / (station_count + 1)
        if curved:
            x, y = _leaf_centerline_point(progress, blade_length, curve_amount)
            tangent = math.degrees(
                math.atan2(
                    blade_length,
                    curve_amount * math.pi * math.cos(math.pi * progress),
                )
            )
        else:
            x, y = 0, blade_length * progress
            tangent = 90

        turtle.penup()
        turtle.goto(blade_start[0] + x, blade_start[1] + y)
        turtle.setheading(tangent)
```

```
        turtle.pendown()
        vein_length = blade_length * 0.34 * math.sin(math.pi * progress)
        _draw_leaf_vein(vein_length, tangent, -1, recursion_level, curved)
        _draw_leaf_vein(vein_length, tangent, 1, recursion_level, curved)

    turtle.penup()
    turtle.goto(blade_start[0] + curve_amount * math.sin(math.pi), blade_start[1] +
blade_length)
    turtle.setheading(90)
    turtle.pendown()


def draw_leaf_straight(
    recursion_level: int,
    petiole_length: float,
    start_position: tuple[float, float] = (0, 0),
) -> None:
    '''draw a vertical, straight-midrib leaf with recursively branching veins'''
    _draw_leaf(recursion_level, petiole_length, curved=False, start_position=start_position)


def draw_leaf_curved(
    recursion_level: int,
    petiole_length: float,
    start_position: tuple[float, float] = (0, 0),
) -> None:
    '''draw a vertical leaf with a curved midrib and curved recursive veins'''
    _draw_leaf(recursion_level, petiole_length, curved=True, start_position=start_position)


class ParkingPermit:
    '''Base class for parking permits.
    attributes:
        owner_name (str): name of permit owner
        months_purchased (int): number of months the permit is valid for
        fee_schedule (dict): dictionary of fees associated with the permit
    methods:
        cost() -> float: calculates the total cost of the permit based on fee schedule
        __str__() -> str: returns a string representation of the permit
        __repr__() -> str: returns a detailed string representation of the permit
    '''
    def __init__(self, owner_name: str, months_purchased: int, fee_schedule: dict) -> None:
        '''initializes a ParkingPermit instance.
        parameters:
            owner_name (str): name of permit owner
            months_purchased (int): number of months the permit is valid for
            fee_schedule (dict): dictionary of fees associated with the permit
        '''
        self.owner_name = owner_name
        self.months_purchased = months_purchased
        self.fee_schedule = fee_schedule
```

```
    def cost(self) -> float:
        '''calculates the total cost of the permit based on the fee schedule.
        parameters:
            none
        returns:
            float - the total cost of the permit'''
        total = 0
        for fee_type, fee_amount in self.fee_schedule.items():
            total += fee_amount
        return total

    def __str__(self) -> str:
        '''returns a string representation of the permit.
        parameters:
            none
        returns:
            str - a string representation of the permit'''
        return f"ParkingPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule})"

    def __repr__(self) -> str:
        '''returns a detailed string representation of the permit.
        parameters:
            none
        returns:
            str - a detailed string representation of the permit'''
        return f"ParkingPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule})"

class StudentPermit(ParkingPermit):
    '''class representing a student parking permit
    inherits from ParkingPermit
    attributes:
        num_vehicles (int): number of vehicles covered by the student permit'''
    def __init__(self, owner_name: str, months_purchased: int, fee_schedule: dict,
num_vehicles: int) -> None:
        '''initializes a StudentPermit instance.
        parameters:
            owner_name (str): name of the permit owner
            months_purchased (int): number of months the permit is valid
            fee_schedule (dict): dictionary of fees associated with the permit
            num_vehicles (int): number of vehicles covered by the student permit
        returns:
            none'''
        super().__init__(owner_name, months_purchased, fee_schedule)
        self.num_vehicles = num_vehicles

    def cost(self) -> float:
        '''calculates the total cost of the student permit.
```

```
        parameters:
            none
        returns:
            float - total cost of the student permit'''
        total = super().cost()
        total += 25 * self.months_purchased
        if self.num_vehicles > 1:
            total += 10 * (self.num_vehicles - 1)
        return total

    def __str__(self) -> str:
        '''returns a string representation of the student permit.
        parameters:
            none
        returns:
            str - a string representation of the student permit'''
        return f"StudentPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule},
num_vehicles={self.num_vehicles})"

    def __repr__(self) -> str:
        '''returns a detailed string representation of the student permit.
        parameters:
            none
        returns:
            str - a detailed string representation of the student permit'''
        return f"StudentPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule},
num_vehicles={self.num_vehicles})"

class FacultyPermit(ParkingPermit):
    '''class representing a faculty parking permit
    inherits from ParkingPermit
    attributes:
        garage_access (bool): indicates if the faculty permit includes garage access'''
    def __init__(self, owner_name: str, months_purchased: int, fee_schedule: dict,
garage_access: bool) -> None:
        '''initializes a FacultyPermit instance.
        parameters:
            owner_name (str): name of the permit owner
            months_purchased (int): number of months the permit is valid
            fee_schedule (dict): dictionary of fees associated with the permit
            garage_access (bool): indicates if the faculty permit includes garage access
        returns:
            none'''
        super().__init__(owner_name, months_purchased, fee_schedule)
        self.garage_access = garage_access

    def cost(self) -> float:
        '''calculates the total cost of the faculty permit.
```

```
        parameters:
            none
        returns:
            float - total cost of the faculty permit'''
        total = super().cost()
        total += 40 * self.months_purchased
        if self.garage_access:
            total += 15 * self.months_purchased
        return total

    def __str__(self) -> str:
        '''returns a string representation of the faculty permit.
        parameters:
            none
        returns:
            str - a string representation of the faculty permit'''
        return f"FacultyPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule},
garage_access={self.garage_access})"

    def __repr__(self) -> str:
        '''returns a detailed string representation of the faculty permit.
        parameters:
            none
        returns:
            str - a detailed string representation of the faculty permit'''
        return f"FacultyPermit(owner_name={self.owner_name},
months_purchased={self.months_purchased}, fee_schedule={self.fee_schedule},
garage_access={self.garage_access})"

def main():
    '''main function to demonstrate the use of StudentPermit and FacultyPermit classes.'''
    test_strB()
    test_Cnk_m()
    test_make_pairs()
    student_permit = StudentPermit("Abby West", 6, {"monthly": 50}, 1)
    faculty_permit = FacultyPermit("Stella Rose", 12, {"monthly": 100}, True)

    print("name: Gigi Bernstein")
    print("Z number: Z23652485")

    print(student_permit)
    print(f"student permit cost: {student_permit.cost()}")

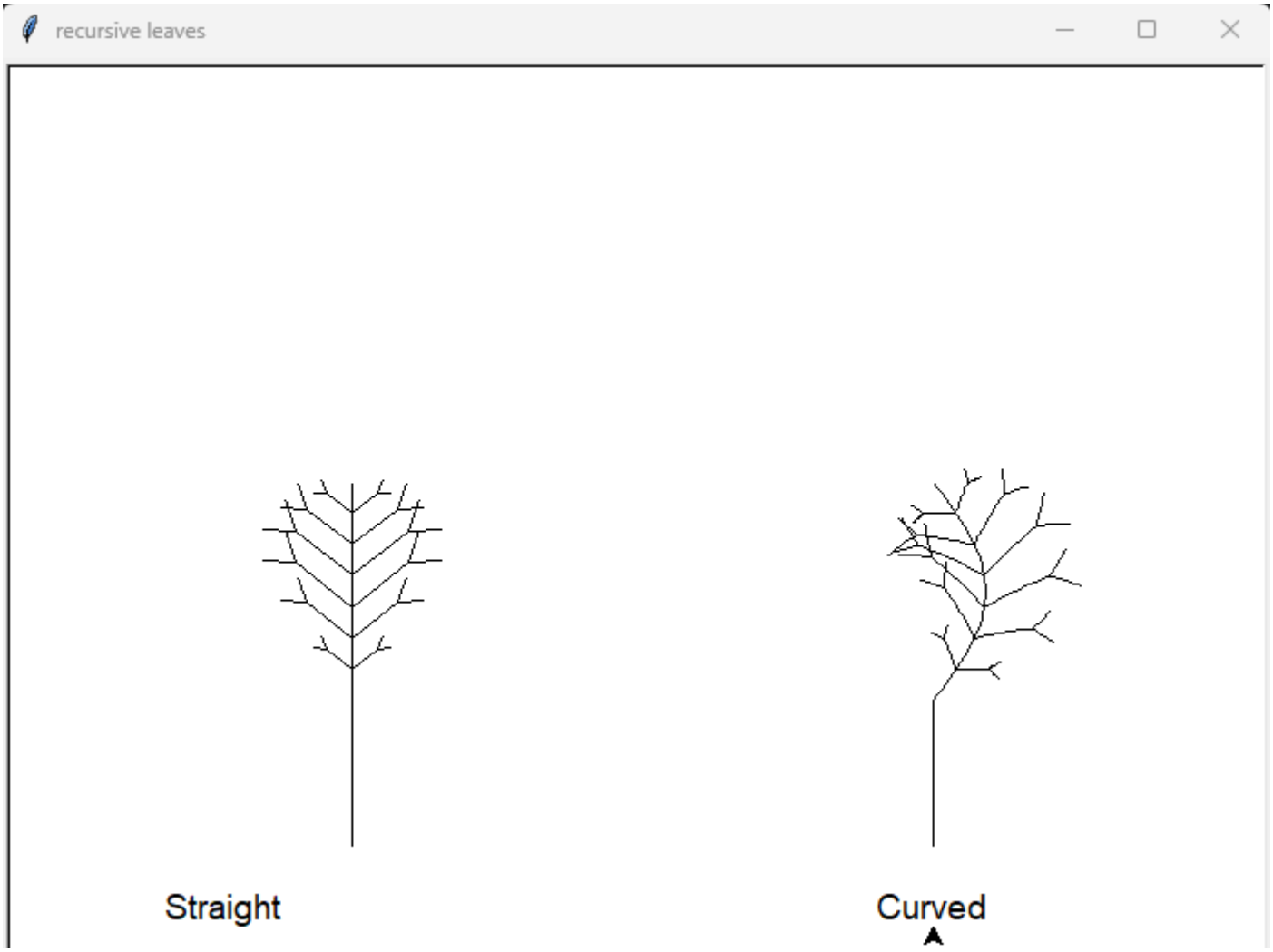
    print(faculty_permit)
    print(f"faculty permit cost: {faculty_permit.cost()}")

    print("\nleaf drawings:")
    turtle.setup(width=700, height=500)
    turtle.title("recursive leaves")
```

```
    draw_leaf_straight(2, 80, (-160, -180))
    draw_leaf_curved(2, 80, (160, -180))
    turtle.penup()
    turtle.goto(-230, -225)
    turtle.write("Straight", align="center", font=("Arial", 14, "normal"))
    turtle.goto(160, -225)
    turtle.write("Curved", align="center", font=("Arial", 14, "normal"))
    turtle.done()

if __name__ == "__main__":
    main()
```

```
PS C:\Users\ginab\Documents\Fall2026\Python\Projects\Homework3> & C:\Users\gigib\AppData\Local\Python\pythoncor
4\python.exe c:/Users/ginab/Documents/Fall2026/Python/Projects/Homework3/P2_Bernstein_Gigi.py
Success: strB zero; returned 0
Success: strB decimal; returned 1234
Success: strB hexadecimal; returned 4D2
Success: strB base 26; returned AA44A1
Success: Cnk_m zero; returned 1
Success: Cnk_m k zero; returned 1
Success: Cnk_m k equals n; returned 1
Success: Cnk_m recursive; returned 155117520
Success: make_pairs equal; paired all values
Success: make_pairs shorter second; stopped at second list
Success: make_pairs shorter first; stopped at first list
Success: make_pairs empty first; returned empty list
Success: make_pairs empty second and both; returned empty lists
name: Gigi Bernstein
Z number: Z23652485
StudentPermit(owner_name=Abby West, months_purchased=6, fee_schedule={'monthly': 50}, num_vehicles=1)
student permit cost: 200
FacultyPermit(owner_name=Stella Rose, months_purchased=12, fee_schedule={'monthly': 100}, garage_access=True)
faculty permit cost: 760
```



Problem 6

```
"""Estimate storage used by a tree of files and folders."""

from __future__ import annotations

import unittest


class Entry:
    """Base class for a named filesystem entry."""

    def __init__(self: Entry, name: str, parent: Entry | None = None) -> None:
        """Initialize an entry with its name and optional parent."""
        self.name: str = name
        self._parent: Entry | None = parent

    def parent(self: Entry) -> Entry | None:
        """Return this entry's parent, or None when it is a root entry."""
        return self._parent

    def size(self: Entry) -> int:
        """Return the entry's size in bytes."""
        raise NotImplementedError("Entry subclasses must implement size().")

    def absolute_path(self: Entry) -> list[Entry]:
        """Return entries from this entry up to and including the root."""
        if self._parent is None:
            return [self]
        return [self, *self._parent.absolute_path()]


class File(Entry):
    """A file with a fixed, nonnegative storage size."""

    def __init__(self: File, name: str, bytes_used: int, parent: Entry | None = None) ->
None:
        """Initialize a file and reject negative storage sizes."""
        super().__init__(name, parent)
        if bytes_used < 0:
            raise ValueError("bytes_used must be nonnegative.")
        self.bytes_used: int = bytes_used

    def size(self: File) -> int:
        """Return the number of bytes used by this file."""
        return self.bytes_used


class Folder(Entry):
    """A folder containing files and other folders."""
```

```
    def __init__(self: Folder, name: str, parent: Entry | None = None) -> None:
        """Initialize an empty folder."""
        super().__init__(name, parent)
        self._children: list[Entry] = []

    def add(self: Folder, entry: Entry) -> None:
        """Add an entry as a child and set its parent to this folder."""
        entry._parent = self
        self._children.append(entry)

    def size(self: Folder) -> int:
        """Return the combined size of all files in this folder tree."""
        return sum(entry.size() for entry in self._children)

    def children(self: Folder) -> tuple[Entry, ...]:
        """Return this folder's direct children as an immutable tuple."""
        return tuple(self._children)


def find_large_files(entry: Entry, minimum_size: int) -> list[File]:
    """Find files at least minimum_size bytes below entry.

    If entry is a File, return that file when it meets the threshold;
    if entry is a Folder, search all of its descendants recursively.
    """
    if isinstance(entry, File):
        return [entry] if entry.size() >= minimum_size else []
    if isinstance(entry, Folder):
        return [
            file
            for child in entry.children()
            for file in find_large_files(child, minimum_size)
        ]
    return []


class EntryHierarchyTests(unittest.TestCase):
    """Tests for recursive size, search, and absolute-path behavior."""

    def setUp(self: EntryHierarchyTests) -> None:
        """Create a three-level nested tree for each test."""
        self.root = Folder("root")
        self.archive = Folder("archive")
        self.department = Folder("department")
        self.lab = Folder("lab")
        self.root.add(self.archive)
        self.archive.add(self.department)
        self.department.add(self.lab)
        self.root.add(File("readme.txt", 300))
```

```
        self.archive.add(File("dataset.csv", 1800))
        self.department.add(File("notes.txt", 900))
        self.lab.add(File("simulation.bin", 6400))
        self.lab.add(File("image.raw", 2100))

    def test_find_large_files_in_folder_at_two_thresholds(self: EntryHierarchyTests) -> None:
        """Return all nested files meeting each requested threshold."""
        self.assertEqual(
            {file.name for file in find_large_files(self.root, 1000)},
            {"dataset.csv", "simulation.bin", "image.raw"},
        )
        self.assertEqual(
            {file.name for file in find_large_files(self.root, 2500)},
            {"simulation.bin"},
        )

    def test_find_large_files_when_entry_is_file(self: EntryHierarchyTests) -> None:
        """Search a file entry itself, including threshold boundaries."""
        large_file = self.lab.children()[0]
        self.assertIsInstance(large_file, File)
        self.assertEqual(find_large_files(large_file, 6400), [large_file])
        self.assertEqual(find_large_files(large_file, 6401), [])

    def test_absolute_path_runs_from_entry_to_root(self: EntryHierarchyTests) -> None:
        """Return the current entry followed by each ancestor."""
        simulation_file = self.lab.children()[0]
        self.assertEqual(
            [entry.name for entry in simulation_file.absolute_path()],
            ["simulation.bin", "lab", "department", "archive", "root"],
        )

    def test_root_absolute_path_contains_only_root(self: EntryHierarchyTests) -> None:
        """Return a one-entry path for a root folder."""
        self.assertEqual(self.root.absolute_path(), [self.root])


def build_example_hierarchy() -> Folder:
    """Build and return a sample hierarchy with four folder levels."""
    root = Folder("university")
    archive = Folder("archive")
    department = Folder("physics")
    lab = Folder("research-lab")
    root.add(archive)
    archive.add(department)
    department.add(lab)
    root.add(File("catalog.txt", 300))
    archive.add(File("enrollment.csv", 1800))
    department.add(File("schedule.txt", 900))
    lab.add(File("simulation.bin", 6400))
    lab.add(File("image.raw", 2100))
```

```
    return root


def demonstrate() -> None:
    """Print recursive size, searches at two thresholds, and one path."""
    root = build_example_hierarchy()
    print(f"Total storage: {root.size()} bytes")
    for minimum_size in (1000, 2500):
        names = [file.name for file in find_large_files(root, minimum_size)]
        print(f"Files at least {minimum_size} bytes: {names}")
    example_file = find_large_files(root, 6400)[0]
    print("Absolute path:", [entry.name for entry in example_file.absolute_path()])


if __name__ == "__main__":
    unittest.main(exit=False)
    demonstrate()
```

Chat

File and folder storage estimation

For this problem, implement the specified constructors, use inheritance and method overriding, and
demonstrate polymorphism by processing objects through a common base-class interface. Assume
nested structures contain no cycles. Write docstrings for classes, methods and functions. Write full
function type annotations.
A university needs to estimate the storage size used by folders containing files and other folders on its
storage system.
a) Define a base class Entry with a name string, a parent Entry attribute, a parent() method, and
a size() method. Define child class File(Entry) with a nonnegative bytes_used attribute; override size() to return that value.
b) Define Folder(Entry) with a list of Entry children and an add(entry) method that adds a child entry to its list of children. Override size() so that it returns the total size of all children,
including those in nested folders. Define a method children() that returns a tuple with all children
entries.
c) Write a recursive method absolute_path() in the Entry class that returns the absolute path of
that entry as a list of Entry objects, starting with the current entry, then its parent, and so on, ending
with the root entry, at the top-most level.
d) Write a recursive find_large_files(entry, minimum_size) function outside of any class that returns all File objects whose sizes are at least minimum_size starting from entry.
Specify its behavior when entry is a file and when it is a folder.
e) Write unit tests for find_large_files() and absolute_path() using a standard Python unit test module. Make sure all unit tests pass.

Todos (3/3)

P6_Bernstein_Gigi.py

f) Construct a folder at least three levels deep. Show that one call to size() on the outer folder
handles both files and folders, and test find_large_files at two thresholds.
g) Use the entry hierarchy from part f). Show how absolute_path() works.

Completed 15 steps in 1m 40s

Implemented the hierarchy, recursive search and path methods, unit tests, and runnable example in P6_Bernstein_Gigi.py. The parent reference is stored internally as `_parent` and returned by `parent()`, since an attribute and method cannot share the same name.

Running the file passes all 4 tests. The example reports a total size of 11,500 bytes, finds the expected files at thresholds of 1,000 and 2,500 bytes, and prints the nested file path up to the root. Pylance reports no errors.

Todos (3/3)

P6_Bernstein_Gigi.py

```
PS C:\Users\ginab\Documents\Fall2026\Python\Projects\Homework3> & C:\Users\gigib\AppData\Local\Python\pythcore-3.14-64\python.exe c:/Users/ginab/Documents/Fall2026/Python/Projects/Homework3/P6_Bernstein_Gigi.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.003s

OK
Total storage: 11500 bytes
Files at least 1000 bytes: ['simulation.bin', 'image.raw', 'enrollment.csv']
Files at least 2500 bytes: ['simulation.bin']
Absolute path: ['simulation.bin', 'research-lab', 'physics', 'archive', 'university']
PS C:\Users\ginab\Documents\Fall2026\Python\Projects\Homework3>
```